Contents

SPECIFICATION — Hybrid Mortgage Calculator MVP (deterministic finance + LLM interface, CLI, uv)

- **Status:** v0.2 — SPEC_REVIEW findings F-001..F-013 integrated
- **Language:** Python 3.12 | `uv` | deterministic calculation core | structured LLM adapter | CLI labels
- **Curriculum source:** `supplemental_docs/hybrid_mortgage_calculator_mvp.md` (§A.1 Purpose, §A.2 Why This Is a Hybrid AI Application, §A.3 MVP Scope, §A.4 The Four Core Calculations, §A.5 Input Model, §A.6 Calculation Engine, §A.7 Financial Validation, §A.8 Natural-Language Interface, §A.9 Tool Interface, §A.10 System Prompt, §A.11 Con-

versational Examples, §A.12 Amortization, §A.13 Architecture, §A.14 Model Independence, §A.15 Testing Strategy, §A.16 Property-Based Testing, §A.17 LLM Evaluation, §A.18 Error Taxonomy, §A.19 Guardrails, §A.20 Precision and Rounding, §A.21 Scope Boundaries, §A.22 Suggested Project Structure, §A.23 MVP Development Sequence, §A.24 What Makes This an AI Engineering Project?, §A.25 Extensions, §A.26 The Central Design Lesson).

- **Scope of this document:** The authoritative contract for a fixed-rate, monthly-payment mortgage calculator with four inverse calculations, amortization, validation, a structured calculator tool, and an optional natural-language adapter. It owns deterministic financial behavior and the model boundary; it does not specify lender underwriting or production financial advice.
- **Normative language:** MUST/MUST NOT/SHALL/SHALL NOT = normative; SHOULD = strong; MAY = optional.
- **Principle: LLM = interpreter and explainer; calculator = authority.** Probabilistic language behavior MUST terminate at a validated, typed calculator request; all financial arithmetic MUST occur in deterministic code.

---

0. Intent and purpose

This lab turns the mortgage-calculator appendix into a small, testable hybrid AI system. It demonstrates that a natural-language interface does not make deterministic arithmetic an LLM responsibility.

The central thesis is:

> **The AI should orchestrate the work that requires intelligence, while conventional software performs the work for which conventional software is better suited.**

The system accepts either a structured request or a natural-language question. Structured requests enter the deterministic calculation core directly. Natural-language requests pass through an `LLMAdapter`, which extracts parameters and intent into a typed request. The adapter MUST call the calculator tool for every numerical result and MUST NOT calculate mortgage values in prose or model-generated code.

Relationship to the source appendix. This lab operationalizes §A.1–§A.26 as executable contracts. The appendix's four inverse relationships become `CalculationRequest`/`CalculationResult`; its validation rules become structured errors; its LLM guidance becomes an adapter contract; and its testing discussion becomes acceptance criteria.

Curriculum mapping. §A.1–§A.3 establish purpose and MVP boundaries. §A.4–§A.7 define the deterministic engine. §A.8–§A.11 define language interpretation, tool use, and clarification. §A.12 defines amortization. §A.13–§A.14 define component boundaries and model substitution. §A.15–§A.20 define testing, evaluation, safety, and precision. §A.21–§A.26 define exclusions, sequencing, and the architectural lesson.

Non-goals: adjustable-rate or interest-only loans, balloon or negative-amortization loans, refinancing or closing-cost analysis, taxes, insurance, HOA fees, lender-specific fees, tax deductions, jurisdiction-specific regulation, lender quotes, autonomous financial advice, and a production-grade hosted model service.

---

1. Actors and goals

Actor	Goals
CLI user (`cli.py`)	Supply three mortgage quantities, request the missing fourth, optionally request an amortization schedule, and receive deterministic JSON or human-readable output.

Actor	Goals
Application service (`service.py`)	Accept a typed request, invoke exactly one calculation, return a typed result or structured error, and never perform model-specific branching.
Calculation core (`calculator.py`)	Validate normalized inputs and calculate exactly one missing quantity using deterministic formulas or a bounded numerical solver.
Amortization engine (`amortization.py`)	Produce a deterministic payment-by-payment schedule from a validated principal, periodic rate, payment count, and payment.
LLM adapter (`llm.py`)	Interpret natural language, normalize units, request clarification when required, call the calculator tool, and explain only the returned result. It is the only probabilistic component.
Calculator tool (`tool.py`)	Expose the calculation core as a pinned structured contract to the LLM adapter.
Verifier/test suite (`tests/`)	Prove formula correctness, inverse relationships, validation, precision, error semantics, tool boundaries, and adapter behavior without requiring a network or model.
External model provider *(optional)*	Return a structured interpretation or explanation. It is not authoritative for arithmetic and is replaceable by an offline mock.

---

2. Requirements (intent, high level)

ID	Statement
R-01	The system MUST support fixed-rate mortgages with monthly payments and MUST calculate exactly one missing value among principal `P`, periodic interest rate `r`, payment count `n`, and periodic payment `M`.
R-02	The system MUST accept canonical normalized units: principal and payment as positive currency amounts, periodic rate as a non-negative decimal, and payment count as a positive integer.
R-03	The system MUST implement payment, principal, payment-count, and interest-rate calculations as deterministic functions independent of the LLM adapter.
R-04	The system MUST validate that exactly three of the four primary quantities are supplied; zero, one, two, or four supplied quantities MUST be rejected.
R-05	The system MUST expose structured calculation inputs, outputs, and errors suitable for direct CLI use and tool calling.
R-06	The system MUST convert annual rates to monthly rates and years to monthly payments before invoking the calculation core.
R-07	The system MUST reject a payment that is not greater than first-period interest when solving for the number of payments.
R-08	The interest-rate solver MUST be bounded by a defined interval, tolerance, and maximum iteration count, and MUST return a structured convergence error when it cannot solve.
R-09	The system MUST generate a deterministic amortization schedule, including payment, principal, interest, and remaining balance for each period.
R-10	The system MUST preserve full internal numeric precision and MUST round only at the presentation boundary.

ID	Statement
R-11	The LLM adapter MUST represent interpretation output as a typed request before any calculation occurs.
R-12	The LLM adapter MUST call the calculator tool for every numerical answer and MUST NOT independently perform mortgage arithmetic.
R-13	The LLM adapter MUST ask for clarification rather than inventing a rate, term, payment, or other financial parameter.
R-14	The system MUST distinguish principal-and-interest payment from taxes, insurance, HOA fees, and other excluded housing costs.
R-15	The LLM adapter MUST be replaceable without changing the calculation core or calculator-tool contract.
R-16	The automated suite MUST run offline with an in-repository mock adapter and MUST NOT require a model API key, network access, or a running model server.
R-17	The CLI MUST provide a direct structured-calculation mode and a natural-language mode; both modes MUST return the same result for equivalent normalized inputs.
R-18	The system MUST classify failures as validation, unsupported scope, clarification-required, solver/convergence, tool, or model/explanation failures.
R-19	The CLI MUST provide an explicit disclaimer that results are estimates for principal and interest only and are not lender-specific quotes or financial advice.
R-20	The project MUST be reproducible with Python 3.12 and `uv`, and its implementation MUST follow the module boundaries and test obligations in this specification.
R-21	The project MUST provide a PyQt5 desktop UI through `mortgage-gui` with calculator and natural-language modes, result metrics, validation/clarification states, and optional amortization display.
R-22	The CLI and GUI MUST provide opt-in levelled diagnostics: omitted verbosity is quiet, bare `--verbose`/`INFO` emits metadata, and `DEBUG` additionally emits raw model prompts/responses; diagnostics MUST NOT alter calculation results or normal result output. CLI diagnostics MUST go to stderr and GUI diagnostics MUST appear in a dedicated status label.
R-23	When Ollama is selected in the GUI, the application MUST query `/api/tags` off the Qt main thread and display the locally available model names in a sorted dropdown; refresh failures MUST be visible without crashing the UI.

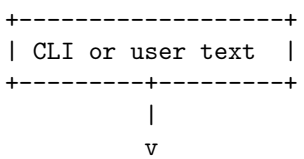
3. Behavior and state model

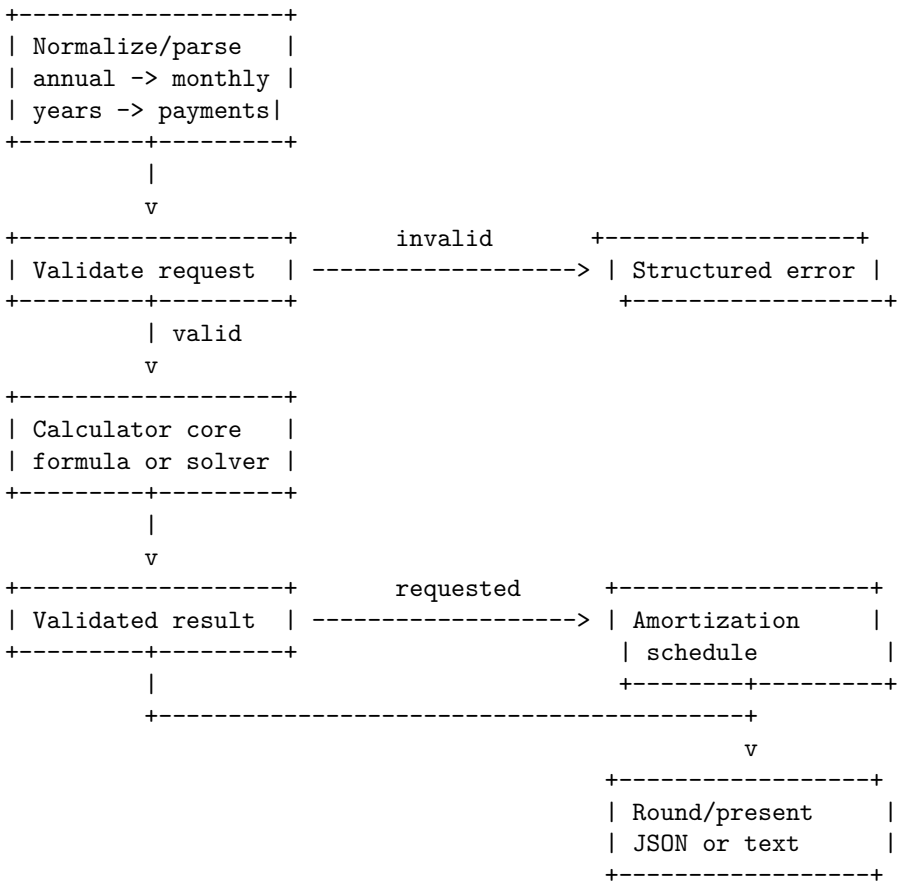
3.1 Lifecycle scope

One calculation request is one stateless operation. It consists of input normalization, validation, calculation, optional amortization generation, result serialization, and presentation. No user or model conversation history is required for v0.1.

A natural-language request has an additional interpretation phase. The adapter may request clarification, but it MUST NOT invoke the calculator until the request has exactly three known primary quantities and explicit assumptions.

3.2 Executable flow





For natural language, the entry point is `LLMAdapter.interpret(text)`. Its output is a `CalculationRequest`, a `ClarificationRequest`, or a structured adapter error. The calculator core is the sole authority for numeric output.

3.3 State model

State	Meaning	Terminal?
`RECEIVED`	Raw CLI arguments or user text received.	no
`NORMALIZED`	Human units converted into canonical units.	no
`VALIDATED`	Exactly three primary values and all domain constraints pass.	no
`CALCULATED`	Missing value calculated successfully.	no
`AMORTIZED`	Optional schedule generated successfully.	no
`PRESENTED`	Result serialized and shown to the caller.	yes
`CLARIFICATION_REQUIRED`	Natural-language input lacks required information or has an ambiguity.	yes
`REJECTED`	Validation, scope, or solver failure returned as a structured error.	yes

The state machine MUST be monotonic within one request: no state after `CALCULATED` may mutate the calculation result, and a failed request MUST NOT produce a partial success object.

---

4. Interfaces / contracts

C-01 Canonical calculation request

```python
@dataclass(frozen=True)
class CalculationRequest:
    principal: Decimal | None       # P; positive currency amount
    periodic_rate: Decimal | None   # r; monthly decimal, >= 0
    payments: int | None            # n; positive integer
    payment: Decimal | None         # M; positive currency amount
    include_schedule: bool = False
    rounding_places: int = 2
```

Exactly one of `principal`, `periodic_rate`, `payments`, and `payment` MUST be `None`. The request MUST use monthly `periodic_rate`; annual rates and years are adapter/CLI concerns. `Decimal` values are canonical internally; JSON serialization is defined by C-09.

C-02 Calculation result

```python
@dataclass(frozen=True)
class CalculationResult:
    principal: Decimal
    periodic_rate: Decimal
    payments: int
    payment: Decimal
    annual_rate: Decimal
    term_years: Decimal
    total_paid: Decimal
    total_interest: Decimal
    missing_quantity: Literal["principal", "periodic_rate", "payments", "payment"]
    schedule: tuple["AmortizationRow", ...] | None = None
```

`total_paid` and `total_interest` MUST be computed from unrounded internal values. Presentation MAY round them. `annual_rate` MUST equal `periodic_rate * 12`, and `term_years` MUST equal `payments / 12`.

C-03 Structured error

```python
@dataclass(frozen=True)
class CalculationError:
    code: Literal[
        "INVALID_QUANTITY_COUNT", "INVALID_PRINCIPAL", "INVALID_RATE",
        "INVALID_PAYMENTS", "INVALID_PAYMENT", "PAYMENT_TOO_LOW",
        "SOLVER_CONVERGENCE", "UNSUPPORTED_SCOPE", "CLARIFICATION_REQUIRED",
        "TOOL_ERROR", "MODEL_ERROR"

$$
    message: str
    parameter: str | None = None
    details: dict[str, str] = field(default_factory=dict)
```

Internal calculator functions MAY raise a private typed exception, but the public `service.py` and `tool.py` interfaces MUST convert it to the discriminated `{ok,result,error,metadata}` response envelope. The error MUST preserve `code`, `message`, and `parameter`; it MUST NOT be a natural-language-only failure.

C-04 Calculation-core interface

```python
class MortgageCalculator(Protocol):
    def calculate(self, request: CalculationRequest) -> CalculationResult: ...
```

The implementation MUST expose pure functions or a stateless service for:

```python
def calculate_payment(principal: Decimal, periodic_rate: Decimal, payments: int) -> Decimal: ...
def calculate_principal(payment: Decimal, periodic_rate: Decimal, payments: int) -> Decimal: ...
def calculate_payments(principal: Decimal, periodic_rate: Decimal, payment: Decimal) -> int: ...
def calculate_rate(principal: Decimal, payment: Decimal, payments: int, *, tolerance: Decimal, max_itera
```

For `periodic_rate == 0`, the core MUST use `payment = principal / payments`, `principal = payment * payments`, and `payments = principal / payment`. The zero-rate payment-count result MUST be accepted only when the quotient is within `integer_tolerance = 1e-9` of an integer; otherwise it MUST return `INVALID_PAYMENTS` with `details["reason"] = "NON_INTEGRAL_TERM"`.

For `periodic_rate > 0`, `calculate_payments` MUST compute the real-valued logarithmic term, then accept it only when it is within `integer_tolerance` of an integer; otherwise it MUST return `INVALID_PAYMENTS` with `details["reason"] = "NON_INTEGRAL_TERM"`. It MUST NOT silently floor or ceil the result.

`calculate_rate` MUST use bisection on `f(r) = calculate_payment(principal, r, payments) - payment` over `[0, 1]`. It MUST evaluate both endpoints, accept an endpoint whose absolute residual is at most `solver_tolerance`, require opposite signs otherwise, select the midpoint each iteration, retain the half-interval containing the sign change, and stop when either `abs(f(mid)) <= solver_tolerance` or interval width is at most `solver_tolerance`. Failure after `solver_max_iterations` MUST return `SOLVER_CONVERGENCE`.

C-05 Calculator tool contract

The tool exposed to an adapter MUST have this logical JSON shape:

```json
{
  "name": "calculate_mortgage",
  "description": "Calculate exactly one missing fixed-rate monthly mortgage parameter.",
  "input": {
    "principal": "number | null",
    "periodic_rate": "number | null",
    "payments": "integer | null",
    "payment": "number | null",
    "include_schedule": "boolean"
  },
  "output": {
    "ok": "boolean",
    "result": "CalculationResult | null",
    "error": "CalculationError | null"
  }
}
```

The tool MUST delegate to the same calculation core used by the CLI. It MUST NOT duplicate formulas.

C-06 Amortization row and schedule

```python
@dataclass(frozen=True)
class AmortizationRow:
    period: int
    payment: Decimal
    principal: Decimal
    interest: Decimal
    balance: Decimal

def amortize(principal: Decimal, periodic_rate: Decimal,
             payments: int, payment: Decimal) -> tuple[AmortizationRow, ...]: ...
```

The final balance MUST be zero within the configured monetary tolerance. Each regular row MUST satisfy `payment = principal + interest`. If the unrounded recurrence leaves a residual balance, the final row MUST set `principal = prior_balance + interest`, `payment = principal + interest`, and `balance = 0`; that row MUST be marked as an adjusted payoff row in the serialized schedule. Intermediate calculations MUST use full precision.

C-07 Language adapter

```python
@dataclass(frozen=True)
class FieldEvidence:
    field: Literal["principal", "periodic_rate", "payments", "payment"]
    source_text: str
    normalized_value: str
    origin: Literal["explicit", "derived"]

@dataclass(frozen=True)
class Interpretation:
    request: CalculationRequest | None
    clarification: str | None
    assumptions: tuple[str, ...]
    evidence: tuple[FieldEvidence, ...]

class LLMAdapter(Protocol):
    def interpret(self, user_text: str) -> Interpretation: ...
    def explain(self, result: CalculationResult, assumptions: tuple[str, ...]) -> str: ...
```

`interpret` MUST return `clarification` instead of guessing when required values are missing or ambiguous. `explain` MUST receive a calculator-produced result; it MUST NOT accept raw numeric claims as an authoritative result.

C-08 Mock adapter

```python
class MockLLMAdapter:
    """Offline adapter with a fixed mapping of canonical example prompts."""
```

The mock MUST support deterministic examples for payment, principal, payment-count, rate, clarification, unsupported scope, and invalid-payment cases. It MUST be used by automated tests and MUST make no network calls.

C-08a Ollama adapter

```python
class OllamaAdapter:
    def __init__(self, model: str = "llama3.2", host: str = "http://localhost:11434", timeout: float = 
    def ask(self, user_text: str) -> AdapterResponse: ...
```

`OllamaAdapter` MUST POST non-streaming JSON to `${host}/api/chat` using the selected `model`. Its first prompt MUST request a JSON-only `Interpretation`; its second prompt MAY request prose explanation, but MUST include the calculator result. `OLLAMA_HOST` and `OLLAMA_MODEL` MAY supply defaults. One request attempt is permitted per phase; timeout, connection, malformed JSON, or missing `message.content` MUST become `MODEL_ERROR`. The interpretation parser MUST accept one optional JSON code fence, normalize local-model `U+2581` space markers, map `loan_amount`/`interest_rate`/`loan_term` aliases, and infer the missing field from explicit user intent when a model incorrectly fills it. No model output may replace or alter the calculator-tool result.

C-08b Ollama model discovery

```
class OllamaClient:
    def list_models(self) -> list[str]: ...
```

`list_models` MUST issue `GET ${host}/api/tags`, read `{ "models": [{"name": "..."}] }`, return unique names sorted lexicographically, and convert network, timeout, malformed JSON, or malformed shape failures to `MODEL_ERROR`. `ModelDiscoveryWorker(QThread)` MUST call it off the Qt main thread. The GUI MUST retain a safe default or display `No local models found` when discovery fails.

C-09 Presentation contract

```
@dataclass(frozen=True)
class PresentationOptions:
    format: Literal["json", "text"] = "text"
    rounding_places: int = 2
    include_schedule: bool = False

@dataclass(frozen=True)
class ResponseMetadata:
    schema_version: str = "0.2"
    adapter: Literal["direct", "mock", "real"] = "direct"
    assumptions: tuple[str, ...] = ()
    calculation_config: dict[str, str] = field(default_factory=dict)
```

JSON output MUST be machine-readable and MUST include either `{ "ok": true, "result": ... }` or `{ "ok": false, "error": ... }`, never both. Every JSON envelope MUST include `metadata` with `schema_version`, `adapter`, `assumptions`, and `calculation_config`. `Decimal` values MUST serialize as base-10 JSON strings; integer counts MUST serialize as JSON integers; NaN and Infinity MUST be rejected.

C-10 GUI contract

`MainWindow` in `ui.py` MUST expose the controls and result widgets named in §5.2, use the service/tool path for calculations, and keep real Ollama requests in an `OllamaWorker(QThread)` rather than the Qt main thread. The GUI MUST show `Calculated`, `Clarification required`, or `Error: <code>` status text and MUST never replace a calculator result with model-generated arithmetic.

---

5. Interface specification

5.1 CLI (`mortgage`), primary surface

Subcommand	Behavior	Exit
`mortgage calculate`	Accept exactly three primary values. `--rate` is a percentage when `--rate-period annual` (default) and a decimal when `monthly`; `--term-years` maps to `payments = term_years * 12`. `--term-years` and `--payments` are mutually exclusive; conflicting aliases are usage errors.	`0` success; `2` usage/validation/clarification; `3` solver/tool failure; `4` unsupported scope.
`mortgage ask TEXT`	Send text to the selected adapter (`--adapter mock | real`); for `real`, use `--model` (default `llama3.2`) and `--host` (default `http://localhost:11434`); print a clarification or invoke the calculator tool and explain its validated result.	`0` success or clarification; `2` usage/validation; `3` solver/tool failure; `4` unsupported scope; `5` model failure.
`mortgage amortize`	Require all four canonical values, or first calculate a missing payment and then schedule it; `--payments` and `--term-years` are mutually exclusive.	`0` success; `2` usage/validation; `3` solver/tool failure.

`--verbose` MAY appear before or after the subcommand. It accepts no value (equivalent to `INFO`) or the explicit levels `INFO` and `DEBUG`. `INFO` MUST emit metadata only; `DEBUG` MUST additionally emit raw model prompts and responses for model-backed operations. Diagnostics MUST go to stderr and stdout MUST be byte-equivalent to a non-verbose successful invocation. `--rate-period` defaults to `annual`. `--rate-period monthly` requires a decimal rate. `--term-years` MUST be a positive integer or a decimal whose multiplication by 12 is an integer; otherwise it is a usage error. The default adapter MUST be `mock`; real model use MUST be opt-in. The CLI MUST print the disclaimer from R-19 in text mode and include a `disclaimer` field in JSON mode. All commands use the same exit partition: `0` success; `2` usage, validation, or clarification; `3` solver/tool failure; `4` unsupported scope; `5` real-model failure.

Representative direct invocation:

```
mortgage calculate --principal 500000 --rate 6.5 --rate-period annual --term-years 30
```

Representative natural-language invocation:

```
mortgage ask "What is the monthly payment on $500,000 at 6.5% for 30 years?"
```

5.2 GUI (`mortgage-gui`), implemented PyQt5 surface

`mortgage-gui` MUST launch a PyQt5 window titled `Hybrid Mortgage Calculator`. The window MUST use a two-column layout: controls on the left and validated results on the right.

The controls MUST provide:

- Mode selector: `Calculator` or `Natural language`.
- Calculator fields for principal, rate, rate units, term years, payments, and payment.
- Natural-language prompt, adapter selector (`Mock` or `Ollama`), a model dropdown populated from Ollama `/api/tags`, a `Refresh models` button, and Ollama host field.
- Amortization toggle, display-precision control, and `Calculate` button.

The result area MUST provide monthly payment, principal, annual rate, term, total paid, total interest, assumptions, status/error or clarification text, a schedule table, and the principal-and-interest disclaimer. Selecting `Ollama` MUST trigger model discovery; the refresh control MUST be disabled while discovery is active and re-enabled when it settles. When `Include schedule` is selected, the UI MUST generate and display a schedule from every successful validated result, including natural-language and rate-solving results. The UI MUST use `service.py/tool.py`, MUST NOT contain financial formulas, and MUST keep Ollama calls off the Qt main thread using a worker. The mock path MUST remain synchronous and offline. The Options panel MUST provide a `Verbosity` selector with `Off`, `INFO`, and `DEBUG`; `INFO` MUST show metadata and `DEBUG` MUST additionally show raw model prompts/responses in the diagnostics label. GUI behavior is covered by T-35..T-39 and is required for v0.2.

5.3 Configuration

The solver defaults MUST be:

```
rate_lower_bound = 0
rate_upper_bound = 1
solver_tolerance = 1e-12
integer_tolerance = 1e-9
solver_max_iterations = 100
monetary_tolerance = 0.01
max_schedule_payments = 1200
```

A real adapter endpoint MAY be configured through `OLLAMA_HOST` or the CLI `--host`; the default is `http://localhost:11434`. `OLLAMA_MODEL` or `--model` selects the model; the default is `llama3.2`. The deterministic core MUST NOT depend on either setting.

---

6. Invariants (must hold in every valid implementation)

ID	Invariant
I-001	Exactly one primary quantity is missing from every accepted `CalculationRequest`.
I-002	All accepted principal, payment, rate, and payment-count values satisfy their domain constraints: `P > 0`, `M > 0`, `r >= 0`, and integer `n > 0`.
I-003	Every successful result is internally consistent: recalculating the supplied relationship from the result reproduces the supplied values within the declared numeric tolerance.
I-004	`periodic_rate` is monthly and `annual_rate` is derived as `periodic_rate * 12`; no layer may silently treat one as the other. CLI annual percentages and monthly decimals MUST be normalized according to §5.1.
I-005	A result marked successful MUST originate from the deterministic calculator core or the calculator tool; model-generated arithmetic alone can never produce success.
I-006	The interest-rate solver uses bisection on `f(r) = calculate_payment(P,r,n) - M` over `[0,1]`; it never executes more than `solver_max_iterations` iterations and never returns a non-finite or out-of-range rate. It stops on `abs(f(mid)) <= solver_tolerance` or interval width `<= solver_tolerance`; otherwise it returns `SOLVER_CONVERGENCE`.
I-007	Internal calculations and amortization use unrounded values; rounding occurs only during presentation.
I-008	An amortization schedule has periods `1..n`, non-negative balances, each row satisfies the payment identity, and a final adjusted payoff row has balance exactly zero within `monetary_tolerance`.
I-009	The LLM adapter has no authority to alter a calculator result, validation error, or excluded-scope decision.

ID	Invariant
I-010	Equivalent structured and natural-language requests produce equivalent canonical requests and equivalent deterministic results.
I-011	The mock adapter and deterministic core perform no network access and are reproducible for identical inputs.
I-012	Every output identifies that the result covers principal and interest only and is not a lender-specific quote.

---

7. Constraints (precise and measurable)

ID	Constraint
K-01	Python MUST be `>=3.12,<3.13`; the project MUST be installable and runnable through `uv`.
K-02	The deterministic core (`calculator.py`, `validation.py`, `amortization.py`, `models.py`) MUST have no imports from an LLM SDK, network client, GUI toolkit, or CLI framework.
K-03	Only `llm.py`/its provider-specific implementation MAY import a model client; `tool.py` MUST call the deterministic core and MUST NOT call a model.
K-04	The rate solver MUST use deterministic bisection on `f(r) = calculate_payment(P,r,n) - M`, interval `[0, 1]`, tolerance `1e-12`, and maximum `100` iterations unless explicitly overridden by typed configuration. It MUST require an exact endpoint or sign change.
K-05	CLI text output MUST round currency to two decimal places by default; JSON MUST encode Decimal values as strings, integer counts as integers, and reject non-finite values.
K-05a	The schedule bound MUST be `max_schedule_payments = 1200` in v0.1/v0.2; larger requests MUST be rejected before schedule allocation.
K-06	The full automated test suite MUST run without network access, API keys, Ollama, or other external services.
K-07	Natural-language mode MUST not invent missing financial parameters; missing or ambiguous inputs MUST yield clarification.
K-08	The implementation MUST not model any scope listed as a non-goal in §0 or §A.21 of the source appendix.
K-09	All subprocess or network behavior used by an optional real adapter MUST have explicit timeout and error conversion; real-adapter failures MUST NOT corrupt deterministic results.
K-10	The amortization engine MUST run in `O(n)` time and MUST reject `payments > max_schedule_payments` where `max_schedule_payments = 1200` before allocating the schedule.

---

8. Edge cases and failure semantics

ID	Case	Semantics
E-01	Zero, one, two, or four primary values supplied	Return `INVALID_QUANTITY_COUNT`; do not calculate or guess the missing field.

ID	Case	Semantics
E-02	Principal or payment is zero/negative	Return `INVALID_PRINCIPAL` or `INVALID_PAYMENT`.
E-03	Periodic rate is negative or non-finite	Return `INVALID_RATE`. A zero rate is valid and MUST use the explicit zero-interest formulas in C-04.
E-04	Payment count is zero, negative, non-integer, or greater than `1200` when a schedule is requested	Return `INVALID_PAYMENTS`. A non-integral inverse term also returns `INVALID_PAYMENTS` with `NON_INTEGRAL_TERM`.
E-05	Payment is less than or equal to `P*r` while solving for payments	Return `PAYMENT_TOO_LOW`; explain that first-period interest is not covered.
E-06	Rate solver reaches its iteration limit or has no bracketed solution	Return `SOLVER_CONVERGENCE` with bounds, endpoint values, tolerance, and iteration details. Exact endpoint roots are accepted.
E-07	Natural-language question omits rate, term, payment, or another required quantity	Return `CLARIFICATION_REQUIRED`; list the missing fields and do not call the calculator.
E-08	Natural-language expression has conflicting units or ambiguous rate period	Return `CLARIFICATION_REQUIRED`; preserve the conflicting interpretations in `details`.
E-09	User asks for taxes, insurance, HOA, lender fees, or an unsupported loan type	Return `UNSUPPORTED_SCOPE`; do not estimate excluded costs.
E-10	Tool receives malformed JSON or a schema-invalid request	Return `TOOL_ERROR`; do not invoke duplicate formulas or fall back to model arithmetic. Tool and service callers MUST receive the discriminated `{ok,result,error,metadata}` envelope rather than an uncaught public exception.
E-11	Real model/network is unavailable, times out, or returns malformed interpretation/explanation	Make at most one bounded adapter attempt, return `MODEL_ERROR` with the provider failure in `details`, and never perform arithmetic fallback. Mock mode remains offline and deterministic.
E-12	Amortization has a residual final balance due to numeric representation	Adjust only the final row within `monetary_tolerance`; preserve the invariant that total principal repaid equals principal.
E-13	`rounding_places` is negative or unreasonably large	Return a CLI usage/validation error before calculation.
E-14	User requests a lender quote or personalized financial advice	Return `UNSUPPORTED_SCOPE` plus the disclaimer; provide no invented lender-specific output.
E-15	CLI supplies conflicting aliases or invalid rate units	Return usage/validation exit `2`; no precedence rule may silently select one value.

9. Acceptance criteria, tests, and evals

9.1 Deterministic formula tests

- **T-01** Payment formula: `P=100000`, annual rate `6%`, `n=360` yields `599.550525152752` within `1e-9`; text output rounds to `$599.55`.
- **T-02** Principal inversion: calculate payment from `(P,r,n)`, then calculate principal from `(M,r,n)`; the result matches `P` within tolerance.
- **T-03** Payment-count inversion: calculate payment from `(P,r,n)`, then calculate payments from `(P,r,M)`; the real-valued result is accepted only within `integer_tolerance=1e-9` of `n`, with no silent floor/ceil.
- **T-04** Rate inversion: calculate payment from `(P,r,n)`, then solve for `r` using the pinned bisection algorithm; the result matches the original periodic rate within `1e-10`, or returns the specified bracket/convergence error.
- **T-05** Zero-interest behavior: `r=0` calculates `M=P/n` and `P=M*n` without division-by-zero errors; `n=P/M` is accepted only for an integer quotient within `1e-9`.

9.2 Validation and failure tests

- **T-06** Every invalid primary-quantity count returns `INVALID_QUANTITY_COUNT` and performs no calculation.
- **T-07** Negative/zero principal, payment, rate, and payment count return the corresponding structured validation error.
- **T-08** `M <= P*r` returns `PAYMENT_TOO_LOW` and includes first-period interest in the error details.
- **T-09** Solver non-convergence respects the maximum iteration count and returns `SOLVER_CONVERGENCE` rather than a numeric guess.
- **T-10** Unsupported taxes, insurance, HOA fees, adjustable-rate loans, and lender quotes return `UNSUPPORTED_SCOPE`.
- **T-10a** Conflicting CLI aliases, invalid annual/monthly rate units, and non-integral `term_years * 12` exit `2` without calculation.

9.3 Amortization tests

- **T-11** A valid schedule contains exactly `n` rows with periods `1..n`.
- **T-12** Each row satisfies `payment = principal + interest` within numeric tolerance.
- **T-13** The balance is non-increasing and the final balance is zero within `monetary_tolerance`.
- **T-14** Total principal repaid equals the starting principal and total interest equals the sum of row interest values within tolerance; an adjusted final payoff row preserves the row identity and has zero balance.

9.4 Contract and boundary tests

- **T-15** The calculator tool and CLI invoke the same calculator implementation; a formula change is observable identically through both paths.
- **T-16** A source/import scan confirms the deterministic core has no LLM, network, GUI, or CLI dependency.
- **T-17** A malformed tool request returns `TOOL_ERROR` and does not produce a result object.
- **T-18** Every success and error presentation includes the principal-and-interest disclaimer.
- **T-18a** JSON Decimal fields are strings, integer counts are integers, metadata contains schema version/adapter/assumptions/config, and non-finite values are rejected.
- **T-19** Equivalent annual-rate/years and monthly-rate/payment-count requests produce equivalent canonical requests.

9.5 Mock natural-language evaluation

- **T-20** The mock adapter extracts payment intent, principal, annual rate, and term from the canonical payment example and calls the tool exactly once.
- **T-21** The mock adapter extracts principal intent from the `$3,000/month at 6% for 30 years` example and returns the deterministic principal result.
- **T-22** The mock adapter extracts payment-count intent and returns `CLARIFICATION_REQUIRED` or `PAYMENT_TOO_LOW` according to the supplied payment.
- **T-23** The mock adapter extracts rate intent and returns the deterministic solver result; its own text arithmetic is ignored.
- **T-24** An underspecified question such as `How much would a $500,000 mortgage cost?` requests rate and term rather than inventing either.
- **T-25** A question about total housing cost distinguishes principal and interest from taxes, insurance, and HOA fees.
- **T-26** Replacing the mock adapter with a second adapter double does not alter calculator outputs for the same typed request.
- **T-26a** A model timeout, malformed interpretation, or explanation failure performs one attempt, returns `MODEL_ERROR`, and never substitutes model-generated arithmetic.
- **T-26b** Every populated interpretation field has `FieldEvidence`; derived fields have `origin="derived"` and a matching assumption.
- **T-34** With a mocked Ollama transport, `OllamaAdapter` sends an interpretation prompt, calls the calculator tool exactly once, sends the calculator result to the explanation phase, and returns `MODEL_ERROR` for malformed interpretation output or transport failure.

9.6 CLI and reproducibility tests

- **T-27** `uv run mortgage calculate ... --format json` emits schema-valid success JSON; invalid invocations exit `2`.
- **T-28** `uv run mortgage ask ... --adapter mock` works with no network, model server, or API key.
- **T-29** Identical mock inputs produce identical canonical result fields and schedule rows.
- **T-30** The CLI text result rounds only presentation values while JSON/canonical results retain enough precision for inverse testing.
- **T-33** With a fixed property-based seed and bounded sample count of 100 valid loans, payment/principal and payment/rate round trips satisfy their declared tolerances; failures identify the generated case.

9.7 Manual smoke evaluation

- **T-31** Run the four conversational examples from §A.11 through `mortgage ask`; confirm tool invocation, clarification, invalid-payment explanation, and scope disclaimer.
- **T-32** Run one direct and one natural-language request with equivalent inputs; compare the displayed payment, total paid, total interest, and optional schedule.

9.8 GUI acceptance (offscreen)

- **T-35** With `QT_QPA_PLATFORM=offscreen`, `mortgage-gui` constructs a window titled `Hybrid Mortgage Calculator` with the required controls, result labels, and schedule table.
- **T-36** Calculator mode submits `P=500000`, annual rate `6.5%`, term `30` and displays a monthly payment containing `$3,160.34` plus the disclaimer.
- **T-37** Invalid calculator input displays an error state and does not fabricate a result.
- **T-38** Mock natural-language mode displays the deterministic calculator result and assumptions without network access.
- **T-39** Ollama mode executes through a worker and leaves the UI thread responsive; a mocked transport failure displays an error state and restores the Calculate button.
- **T-44** A successful natural-language result with `Include schedule` selected displays the full validated amortization schedule, including for a rate-solving request.

- **T-42** A mocked `GET /api/tags` response returns unique lexicographically sorted names and populates the GUI dropdown without a network call.
- **T-43** A mocked discovery failure displays a model-discovery error, leaves a safe dropdown state, and does not crash the GUI.
- **T-40** CLI `--verbose` is accepted before and after a subcommand, writes diagnostics only to stderr, and leaves normal stdout and exit code unchanged.
- **T-41** GUI `Verbosity` defaults to `Off`; `INFO` displays metadata and `DEBUG` displays metadata plus raw model prompts/responses in the dedicated diagnostics label.

---

10. Dependencies and environment

Concern	Decision	Rationale
Python	`>=3.12,<3.13`	Pinned implementation runtime.
Environment	`uv`	Reproducible dependency and command execution.
Numeric representation	Python `Decimal` with an explicit context	Avoid premature currency rounding and make tolerance behavior testable.
CLI	Standard library `argparse` or an equivalent lightweight CLI layer	Direct, offline, and testable.
Schema/tool validation	Standard-library validation or a pinned schema library	Tool requests and JSON outputs require deterministic validation.
Testing	`pytest` and `hypothesis`	Formula, inverse, boundary, and generated-case coverage; T-33 is mandatory.
LLM adapter	Protocol plus `MockLLMAdapter`; optional provider-specific adapter	Keeps model choice outside the deterministic core.
Network/model	None required for tests or mock CLI path	Satisfies offline reproducibility.
GUI	PyQt5 `>=5.15,<6`; `pytest-qt` `>=4.4,<5` in the `gui` extra	Implemented desktop UI and offscreen tests for `mortgage-gui`.

Expected project structure:

```
labs/week4/chapter31/
+-- SPEC.md
+-- pyproject.toml
+-- README.md
+-- src/mortgage/
|   +-- __init__.py
|   +-- models.py
|   +-- validation.py
|   +-- calculator.py
|   +-- amortization.py
|   +-- tool.py
|   +-- llm.py
|   +-- service.py
|   +-- presentation.py
|   +-- cli.py
+-- tests/
    +-- test_calculator.py
    +-- test_validation.py
```

```
    +-- test_amortization.py
    +-- test_tool_contract.py
    +-- test_llm_adapter.py
    +-- test_cli.py
    +-- test_ui.py
```

Reproducibility commands:

```
uv sync
uv run pytest
uv run mortgage calculate --principal 500000 --rate 6.5 --rate-period annual --term-years 30
uv run mortgage ask --adapter mock "What is the payment on a $500,000 mortgage at 6.5% for 30 years?"
```

---

11. Traceability matrix (id -> where realized)

```
R-01/R-03/R-04 --> models.py + calculator.py (four inverse paths and one missing value) --> T-01..T-05,
R-02/R-06      --> validation.py + cli.py (canonical monthly units and normalization) --> T-07, T-19
R-05           --> models.py + tool.py (typed result/error envelope) --> T-15, T-17, T-27
R-07/R-08      --> calculator.py (payment guard and bounded rate solver) --> T-08, T-09
R-09           --> amortization.py (deterministic schedule) --> T-11..T-14
R-10           --> calculator.py + presentation.py (precision boundary) --> T-14, T-30
R-11/R-13      --> llm.py (typed interpretation and clarification) --> T-20..T-25
R-12/R-15      --> llm.py + tool.py (calculator authority and adapter seam) --> T-15, T-26, T-26a
R-14/R-19      --> presentation.py + llm.py (scope boundary and disclaimer) --> T-18, T-25
R-16/R-20      --> pyproject.toml + MockLLMAdapter + tests/ --> T-16, T-28, T-29
R-21           --> ui.py + pyproject.toml (`mortgage-gui`) --> T-35..T-39
R-22           --> cli.py + ui.py (opt-in diagnostics) --> T-40, T-41
R-23           --> llm.py:OllamaClient.list_models + ui.py:ModelDiscoveryWorker --> T-42, T-43
R-17           --> cli.py + service.py (direct and natural-language modes) --> T-27, T-28, T-32
C-10           --> ui.py (PyQt5 two-mode surface and worker) --> T-35..T-39
R-18           --> models.py + validation.py + llm.py (error taxonomy) --> T-06..T-10, T-17, T-26a
I-001/I-002    --> validation.py + CalculationRequest --> T-06, T-07
I-003/I-004    --> calculator.py + normalization --> T-02..T-04, T-19
I-005/I-009    --> tool.py + service.py + adapter protocol --> T-15, T-20, T-26
I-006          --> calculator.py solver configuration --> T-09
I-007          --> calculator.py + presentation.py --> T-14, T-30
I-008          --> amortization.py --> T-11..T-14
I-010/I-011    --> MockLLMAdapter + equivalent-path tests --> T-19, T-26, T-29
I-012          --> presentation.py --> T-18
K-01/K-02      --> pyproject.toml + module imports --> T-16, T-27
K-03/K-06      --> llm.py + MockLLMAdapter --> T-16, T-28
K-04           --> calculator.py solver config --> T-09
K-05/K-07      --> presentation.py + llm.py --> T-18, T-24, T-30
K-08           --> scope policy in llm.py/service.py --> T-10, T-25
K-09           --> optional real adapter boundary --> T-26 (mock replacement contract)
K-10           --> amortization.py safe bound --> T-11
E-01..E-06     --> validation.py + calculator.py --> T-06..T-09
E-07..E-09     --> llm.py + service.py --> T-24, T-25, T-26b
E-10/E-11      --> tool.py + llm.py --> T-17, T-28
E-12           --> amortization.py --> T-13, T-14
E-13/E-14      --> cli.py + scope policy --> T-10, T-27
C-08a          --> llm.py:OllamaClient/OllamaAdapter + cli.py flags --> T-34
C-08b          --> llm.py + ui.py model discovery worker --> T-42, T-43
F-001..F-003 --> §4 C-04 + §5.1 + §5.3 (normalization, zero-rate, bisection) --> T-03..T-05, T-10a
```

```
F-004..F-005 --> C-06 + C-09 (schedule payoff and Decimal JSON) --> T-14, T-18a
F-006..F-008 --> §5.1 + §5.3 + K-05a (bounds, errors, exits) --> T-10a, T-27
F-009..F-011 --> C-07 + C-09 + E-11 (evidence, provenance, model failures) --> T-18a, T-26a, T-26b
F-012..F-013 --> T-01 + T-33 (known oracle and property testing) --> T-01, T-33
```

End of specification. This document is the source of truth; implementation and tests MUST be derived from it and kept synchronized with the traceability matrix.